

Slip1

Q1) Write a C program to perform the following operations using static implementation of stack

- i. Accept n integers from user and push it into the stack.**
- ii. Pop the element from the stack and display it (Use isEmpty())**

```
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

// Push function
void push(int value) {
    if(top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = value;
}

// Pop function
void pop() {
    if(top == -1) { // isEmpty check
        printf("Stack is empty\n");
        return;
    }
    printf("Popped element: %d\n", stack[top--]);
}

int main() {
    int n, value;

    printf("Enter number of elements to push: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        printf("Enter element %d: ", i+1);
        scanf("%d", &value);
```

```

    push(value);
}

printf("\nPopping elements from stack:\n");
while(top != -1) { // while stack is not empty
    pop();
}

return 0;
}

```

Slip 2

Write a menu driven program for the following operations on singly linked list.

- i. Create and display a linked list of n nodes.**
- ii. Search a particular element from a linked list.**

```

#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to create linked list
struct Node* createList(int n) {
    struct Node *head = NULL, *temp = NULL, *newNode = NULL;
    int value;

    for(int i = 0; i < n; i++) {
        printf("Enter data for node %d: ", i+1);
        scanf("%d", &value);

        newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = value;
    }
}

```

```

    newNode->next = NULL;

    if(head == NULL) {
        head = newNode;
        temp = head;
    } else {
        temp->next = newNode;
        temp = newNode;
    }
}
return head;
}

// Function to display linked list
void displayList(struct Node* head) {
    if(head == NULL) {
        printf("Linked list is empty\n");
        return;
    }
    struct Node* temp = head;
    printf("Linked list elements: ");
    while(temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// Function to search an element in linked list
void searchElement(struct Node* head, int key) {
    struct Node* temp = head;
    int pos = 1;
    while(temp != NULL) {
        if(temp->data == key) {
            printf("%d found at position %d\n", key, pos);
            return;
        }
        temp = temp->next;
        pos++;
    }
}

```

```

    printf("%d not found in the list\n", key);
}

int main() {
    struct Node* head = NULL;
    int choice, n, key;

    do {
        printf("\n--- Singly Linked List Menu ---\n");
        printf("1. Create and Display Linked List\n");
        printf("2. Search an Element\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter number of nodes: ");
                scanf("%d", &n);
                head = createList(n);
                displayList(head);
                break;
            case 2:
                if(head == NULL) {
                    printf("List is empty. Create list first.\n");
                    break;
                }
                printf("Enter element to search: ");
                scanf("%d", &key);
                searchElement(head, key);
                break;
            case 3:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice! Try again.\n");
        }
    } while(choice != 3);

    return 0;
}

```

```
}
```

Slip3

Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int n, x, found = 0;

    printf("Enter the size of the array: ");
    scanf("%d", &n);

    int arr[n];

    // Initialize random number generator
    srand(time(0));

    // Fill array with random integers (0-99)
    printf("Random array: ");
    for(int i = 0; i < n; i++) {
        arr[i] = rand() % 100;
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Accept value to search
    printf("Enter value to search: ");
    scanf("%d", &x);

    // Linear search
    for(int i = 0; i < n; i++) {
        if(arr[i] == x) {
            printf("%d found at position %d\n", x, i+1); // 1-based index
            found = 1;
        }
    }
}
```

```

        break;
    }
}

if(!found) {
    printf("%d not found in the array\n", x);
}

return 0;
}

```

Slip 4

Write a 'C' program to reverse a string using Static implementation of Stack

```

#include <stdio.h>
#include <string.h>
#define MAX 100

char stack[MAX];
int top = -1;

// Push character onto stack
void push(char ch) {
    if(top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = ch;
}

// Pop character from stack
char pop() {
    if(top == -1) {
        printf("Stack Underflow\n");
        return '\0';
    }
    return stack[top--];
}

int main() {

```

```

char str[MAX], reversed[MAX];
int len;

printf("Enter a string: ");
fgets(str, MAX, stdin); // read string including spaces
str[strcspn(str, "\n")] = '\0'; // remove newline

len = strlen(str);

// Push all characters onto stack
for(int i = 0; i < len; i++) {
    push(str[i]);
}

// Pop all characters to reverse the string
for(int i = 0; i < len; i++) {
    reversed[i] = pop();
}
reversed[len] = '\0';

printf("Reversed string: %s\n", reversed);

return 0;
}

```

Slip 5

Write a C program to implement singly linked list with Create and Display operation.

```

#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to create a linked list with n nodes

```

```

struct Node* createList(int n)
{
    struct Node *head = NULL, *temp = NULL, *newNode = NULL;
    int value;

    for(int i = 0; i < n; i++)
    {
        printf("Enter data for node %d: ", i+1);
        scanf("%d", &value);

        newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = value;
        newNode->next = NULL;

        if(head == NULL) {
            head = newNode; // first node
            temp = head;
        } else {
            temp->next = newNode;
            temp = newNode;
        }
    }
    return head;
}

// Function to display linked list
void displayList(struct Node* head) {
    if(head == NULL) {
        printf("Linked list is empty\n");
        return;
    }

    struct Node* temp = head;
    printf("Linked list elements: ");
    while(temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

```

```

int main() {
    struct Node* head = NULL;
    int n;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    head = createList(n);
    displayList(head);

    return 0;
}

```

Slip 6

Q 1) Write a menu driven program for the following operations on singly link list.

- i. Create and display a linked list of n nodes.**
- ii. Search an element in a linked list.**

```

#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to create linked list
struct Node* createList(int n) {
    struct Node *head = NULL, *temp = NULL, *newNode = NULL;
    int value;

    for(int i = 0; i < n; i++) {
        printf("Enter data for node %d: ", i+1);
        scanf("%d", &value);

        newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = value;
    }
}

```

```

    newNode->next = NULL;

    if(head == NULL) {
        head = newNode;
        temp = head;
    } else {
        temp->next = newNode;
        temp = newNode;
    }
}
return head;
}

// Function to display linked list
void displayList(struct Node* head) {
    if(head == NULL) {
        printf("Linked list is empty\n");
        return;
    }
    struct Node* temp = head;
    printf("Linked list elements: ");
    while(temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// Function to search an element in linked list
void searchElement(struct Node* head, int key) {
    struct Node* temp = head;
    int pos = 1;
    while(temp != NULL) {
        if(temp->data == key) {
            printf("%d found at position %d\n", key, pos);
            return;
        }
        temp = temp->next;
        pos++;
    }
}

```

```

    printf("%d not found in the list\n", key);
}

int main() {
    struct Node* head = NULL;
    int choice, n, key;

    do {
        printf("\n--- Singly Linked List Menu ---\n");
        printf("1. Create and Display Linked List\n");
        printf("2. Search an Element\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter number of nodes: ");
                scanf("%d", &n);
                head = createList(n);
                displayList(head);
                break;
            case 2:
                if(head == NULL) {
                    printf("List is empty. Create list first.\n");
                    break;
                }
                printf("Enter element to search: ");
                scanf("%d", &key);
                searchElement(head, key);
                break;
            case 3:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice! Try again.\n");
        }
    } while(choice != 3);

    return 0;
}

```

```
}
```

Slip 7

Accept n sorted values in array from user. Accept a value x from user and use binary search algorithm to check whether the number is present in sorted array or not and output the position if the number is present.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, x, low, high, mid, found = 0;
```

```
    printf("Enter size of array: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    // Accept sorted array elements
```

```
    printf("Enter %d sorted elements in ascending order:\n", n);
```

```
    for(int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    printf("Enter value to search: ");
```

```
    scanf("%d", &x);
```

```
    // Binary search
```

```
    low = 0;
```

```
    high = n - 1;
```

```
    while(low <= high) {
```

```
        mid = (low + high) / 2;
```

```
        if(arr[mid] == x) {
```

```
            printf("%d found at position %d\n", x, mid + 1); // 1-based position
```

```
            found = 1;
```

```
            break;
```

```
        } else if(arr[mid] < x) {
```

```
            low = mid + 1;
```

```
        } else {
```

```

        high = mid - 1;
    }
}

if(!found) {
    printf("%d not found in the array\n", x);
}

return 0;
}

```

Slip 8

Sort a random array of n integers (accept the value of n from user) in ascending order by using bubble sort algorithm.

```

#include <stdio.h>

int main() {
    int n, temp;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];

    // Accept array elements from user
    printf("Enter %d elements:\n", n);
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Bubble sort in ascending order
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j+1]
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

```

```

    }
}

// Display sorted array
printf("Sorted array in ascending order: ");
for(int i = 0; i < n; i++) {
    printf("%d ", arr[i]);
}
printf("\n");

return 0;
}

```

Slip 9

Create a random array of n integers. Accept a value x from user and use linear search algorithm to check whether the number is present in the array or not and output the position if the number is present.

i) Accept & Display Function

ii) Linear Search Function

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX 100

// Function to accept and display random array
void acceptDisplay(int arr[], int n) {
    srand(time(0)); // seed for random numbers
    printf("Random array: ");
    for(int i = 0; i < n; i++) {
        arr[i] = rand() % 100; // random numbers 0-99
        printf("%d ", arr[i]);
    }
    printf("\n");
}

// Function to perform linear search
int linearSearch(int arr[], int n, int x) {
    for(int i = 0; i < n; i++) {

```

```

        if(arr[i] == x)
            return i + 1; // 1-based position
    }
    return -1; // not found
}

int main() {
    int n, x, pos;
    int arr[MAX];

    printf("Enter size of array: ");
    scanf("%d", &n);

    acceptDisplay(arr, n);

    printf("Enter value to search: ");
    scanf("%d", &x);

    pos = linearSearch(arr, n, x);

    if(pos != -1)
        printf("%d found at position %d\n", x, pos);
    else
        printf("%d not found in the array\n", x);

    return 0;
}

```

Slip 10

Write a C program to implement Static implementation of stack of integers with following operation. a. Initialize() b. Push() c. Pop() d. Isempty() e. Isfull() f. Peek()

```

#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

```

// Push element

```
void push(int value) {  
    if(top == MAX - 1) {  
        printf("Stack Overflow\n");  
        return;  
    }  
    stack[++top] = value;  
}
```

// Pop element

```
void pop() {  
    if(top == -1) {  
        printf("Stack Underflow\n");  
        return;  
    }  
    printf("Popped element: %d\n", stack[top--]);  
}
```

// Check if stack is empty

```
int isEmpty() {  
    return top == -1;  
}
```

// Check if stack is full

```
int isFull() {  
    return top == MAX - 1;  
}
```

// Peek top element

```
void peek() {  
    if(isEmpty()) {  
        printf("Stack is empty\n");  
        return;  
    }  
    printf("Top element: %d\n", stack[top]);  
}
```

// Display stack

```
void display() {  
    if(isEmpty()) {
```

```

        printf("Stack is empty\n");
        return;
    }
    printf("Stack elements: ");
    for(int i = 0; i <= top; i++)
        printf("%d ", stack[i]);
    printf("\n");
}

int main() {
    int choice, value;

    do {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Check if Empty\n");
        printf("6. Check if Full\n");
        printf("7. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:
                pop();
                break;
            case 3:
                peek();
                break;
            case 4:
                display();
                break;
            case 5:

```

```

        printf(isEmpty() ? "Stack is empty\n" : "Stack is not empty\n");
        break;
    case 6:
        printf(isFull() ? "Stack is full\n" : "Stack is not full\n");
        break;
    case 7:
        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice! Try again.\n");
    }
} while(choice != 7);

return 0;
}

```

Slip 11

Write a 'C' program to reverse a string using Static implementation of Stack.

```

#include <stdio.h>
#include <string.h>
#define MAX 100

int top = -1;
char stack[MAX];

// Push character onto stack
void push(char ch) {
    if(top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = ch;
}

// Pop character from stack
char pop() {
    if(top == -1) {
        printf("Stack Underflow\n");
    }
}

```

```

        return '\0';
    }
    return stack[top--];
}

int main() {
    char str[MAX], reversed[MAX];
    int len;

    printf("Enter a string: ");
    scanf("%s", str); // Using scanf to read string (no spaces allowed)

    len = strlen(str);

    // Push all characters onto stack
    for(int i = 0; i < len; i++) {
        push(str[i]);
    }

    // Pop all characters to reverse the string
    for(int i = 0; i < len; i++) {
        reversed[i] = pop();
    }
    reversed[len] = '\0';

    printf("Reversed string: %s\n", reversed);

    return 0;
}

```

Slip 12

Sort a random array of n integers (accept the value of n from user) in ascending order by using selection sort algorithm.

```

#include <stdio.h>
#include <stdlib.h>

```

```

#include <time.h>

int main() {
    int n, i, j, minIndex, temp;

    printf("Enter the size of the array: ");
    scanf("%d", &n);

    int arr[n];
    srand(time(0));

    // Generate random array
    printf("Random array: ");
    for(i = 0; i < n; i++) {
        arr[i] = rand() % 100; // random numbers 0-99
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Selection sort
    for(i = 0; i < n - 1; i++) {
        minIndex = i;
        for(j = i + 1; j < n; j++) {
            if(arr[j] < arr[minIndex])
                minIndex = j;
        }
        // Swap
        temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }

    // Display sorted array
    printf("Sorted array (ascending order): ");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    return 0;
}

```

Slip 13

Accept n sorted values in array from user. Accept a value x from user and use binary search algorithm to check whether the number is present in sorted array or not and output the position if the number is present.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, x, low, high, mid, found = 0;
```

```
    printf("Enter the number of elements: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    // Accept sorted array elements
```

```
    printf("Enter %d elements in ascending order:\n", n);
```

```
    for(int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    printf("Enter value to search: ");
```

```
    scanf("%d", &x);
```

```
    // Binary search
```

```
    low = 0;
```

```
    high = n - 1;
```

```
    while(low <= high) {
```

```
        mid = (low + high) / 2;
```

```
        if(arr[mid] == x) {
```

```
            printf("%d found at position %d\n", x, mid + 1); // 1-based position
```

```
            found = 1;
```

```
            break;
```

```
        } else if(arr[mid] < x) {
```

```
            low = mid + 1;
```

```
        } else {
```

```
            high = mid - 1;
```

```

    }
}

if(!found) {
    printf("%d not found in the array\n", x);
}

return 0;
}

```

Slip 14

Read the data from “Student.txt (Stud_name, Age, Percentage)” file and sort on Percentage using selection sort.(Create a separate sorted file).

```

#include <stdio.h>
#define MAX 100

typedef struct {
    char name[50];
    int age;
    float percentage;
} Student;

int main() {
    Student s[MAX], temp;
    int n = 0;

    FILE *fp = fopen("Student.txt", "r");
    if(!fp) { printf("Cannot open file\n"); return 1; }

    // Read data
    while(fscanf(fp, "%s %d %f", s[n].name, &s[n].age, &s[n].percentage) != EOF)
        n++;
    fclose(fp);

    // Selection sort by percentage
    for(int i = 0; i < n-1; i++) {
        for(int j = i+1; j < n; j++) {
            if(s[i].percentage > s[j].percentage) {
                temp = s[i]; s[i] = s[j]; s[j] = temp;
            }
        }
    }
}

```

```

    }
}

// Write sorted data to new file
FILE *fp2 = fopen("Student_sorted.txt", "w");
for(int i = 0; i < n; i++)
    fprintf(fp2, "%s %d %.2f\n", s[i].name, s[i].age, s[i].percentage);
fclose(fp2);

printf("Sorted data saved to Student_sorted.txt\n");
return 0;
}

```

Slip 15

Sort a random array of n integers (accept the value of n from user) in ascending order by using insertion sort algorithm.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int n, i, j, key;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];
    srand(time(0));

    // Generate random array
    printf("Random array: ");
    for(i = 0; i < n; i++) {
        arr[i] = rand() % 100; // random numbers 0-99
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Insertion sort
    for(i = 1; i < n; i++) {
        key = arr[i];

```

```

j = i - 1;

// Move elements greater than key one position ahead
while(j >= 0 && arr[j] > key) {
    arr[j + 1] = arr[j];
    j--;
}
arr[j + 1] = key;
}

// Display sorted array
printf("Sorted array (ascending order): ");
for(i = 0; i < n; i++)
    printf("%d ", arr[i]);
printf("\n");

return 0;
}

```

Slip 16

Q1) Read the data from file 'cities.txt' containing names of 10 cities and their STD codes. Accept a name of the city from user and use linear search algorithm to check whether the name is present in the file and output the STD code, otherwise output “city not in the list”. i) Accept & Display Function ii) Linear Search Function

```

#include <stdio.h>
#include <string.h>

#define MAX 100

typedef struct {
    char name[50];
    int stdCode;
} City;

// Function to accept and display cities from file
int acceptCities(City cities[]) {
    FILE *fp = fopen("cities.txt", "r");
    if(fp == NULL) {
        printf("Error opening file!\n");
    }
}

```

```

    return 0;
}

int n = 0;
while(fscanf(fp, "%s %d", cities[n].name, &cities[n].stdCode) != EOF) {
    n++;
}
fclose(fp);

// Display cities
printf("Cities and their STD codes:\n");
for(int i = 0; i < n; i++) {
    printf("%s -> %d\n", cities[i].name, cities[i].stdCode);
}

return n;
}

// Linear search function
int linearSearch(City cities[], int n, char cityName[]) {
    for(int i = 0; i < n; i++) {
        if(strcmp(cities[i].name, cityName) == 0) {
            return i; // return index if found
        }
    }
    return -1; // not found
}

int main() {
    City cities[MAX];
    int n;

    n = acceptCities(cities);

    char searchCity[50];
    printf("\nEnter city name to search: ");
    scanf("%s", searchCity);

    int index = linearSearch(cities, n, searchCity);
    if(index != -1) {

```

```

        printf("%s found! STD code: %d\n", cities[index].name,
cities[index].stdCode);
    } else {
        printf("City not in the list\n");
    }

    return 0;
}

```

Slip 17

Perform the following operations using implementation of stack (Static or dynamic) i. Accept n integers from user and push it into the stack. ii. Reverses a string of characters.

```

#include <stdio.h>
#include <string.h>
#define MAX 100

char stack[MAX];
int top = -1;

// Push character onto stack
void push(char ch) {
    if(top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = ch;
}

// Pop character from stack
char pop() {
    if(top == -1) {
        printf("Stack Underflow\n");
        return '\0';
    }
    return stack[top--];
}

```

```

int main() {
    char str[MAX], reversed[MAX];
    int len;

    printf("Enter a string: ");
    scanf("%s", str); // input string (without spaces)
    len = strlen(str);

    // Push all characters onto stack
    for(int i = 0; i < len; i++)
        push(str[i]);

    // Pop characters to reverse the string
    for(int i = 0; i < len; i++)
        reversed[i] = pop();
    reversed[len] = '\0';

    printf("Reversed string: %s\n", reversed);

    return 0;
}

```

Slip 18

Sort a random array of n integers (accept the value of n from user) in ascending order by using bubble sort algorithm.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int n, i, j, temp;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

```

```

int arr[n];
srand(time(0));

// Generate random array
printf("Random array: ");
for(i = 0; i < n; i++) {
    arr[i] = rand() % 100; // random numbers 0-99
    printf("%d ", arr[i]);
}
printf("\n");

// Bubble sort
for(i = 0; i < n-1; i++) {
    for(j = 0; j < n-i-1; j++) {
        if(arr[j] > arr[j+1]) {
            // Swap
            temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}

// Display sorted array
printf("Sorted array (ascending order): ");
for(i = 0; i < n; i++)
    printf("%d ", arr[i]);
printf("\n");

return 0;
}

```

Slip 19

Read the data from the file “employee.txt” and sort on age using Bubble sort.

```

#include <stdio.h>
#include <string.h>

```

```

#define MAX 100

typedef struct {
    char name[50];
    int age;
    float salary;
} Employee;

int main() {
    Employee emp[MAX], temp;
    int n = 0;

    FILE *fp = fopen("employee.txt", "r");
    if(fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    // Read data from file
    while(fscanf(fp, "%s %d %f", emp[n].name, &emp[n].age, &emp[n].salary) !=
EOF) {
        n++;
    }
    fclose(fp);

    // Bubble sort by age
    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-i-1; j++) {
            if(emp[j].age > emp[j+1].age) {
                temp = emp[j];
                emp[j] = emp[j+1];
                emp[j+1] = temp;
            }
        }
    }

    // Display sorted data
    printf("Employee data sorted by age:\n");
    printf("Name\tAge\tSalary\n");
    for(int i = 0; i < n; i++) {

```

```

        printf("%s\t%d\t%.2f\n", emp[i].name, emp[i].age, emp[i].salary);
    }

    return 0;
}

```

Text File

```

Alice 28 45000
Bob 35 55000
Charlie 25 40000
David 30 50000

```

Slip 20

Write a C program to implement Static implementation of stack of integers with following operation. a. Initialize() b. Push() c. Pop() d. Isempty() e. Isfull() f. Peek()

```

#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

// Push element
void push(int value) {
    if(top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = value;
}

// Pop element
void pop() {
    if(top == -1) {
        printf("Stack Underflow\n");
        return;
    }
}

```

```
    }  
    printf("Popped element: %d\n", stack[top--]);  
}
```

```
// Check if stack is empty  
int isEmpty() {  
    return top == -1;  
}
```

```
// Check if stack is full  
int isFull() {  
    return top == MAX - 1;  
}
```

```
// Peek top element  
void peek() {  
    if(isEmpty()) {  
        printf("Stack is empty\n");  
        return;  
    }  
    printf("Top element: %d\n", stack[top]);  
}
```

```
// Display stack  
void display() {  
    if(isEmpty()) {  
        printf("Stack is empty\n");  
        return;  
    }  
    printf("Stack elements: ");  
    for(int i = 0; i <= top; i++)  
        printf("%d ", stack[i]);  
    printf("\n");  
}
```

```
int main() {  
    int choice, value;  
  
    do {  
        printf("\n--- Stack Menu ---\n");
```

```
printf("1. Push\n");
printf("2. Pop\n");
printf("3. Peek\n");
printf("4. Display\n");
printf("5. Check if Empty\n");
printf("6. Check if Full\n");
printf("7. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice) {
    case 1:
        printf("Enter value to push: ");
        scanf("%d", &value);
        push(value);
        break;
    case 2:
        pop();
        break;
    case 3:
        peek();
        break;
    case 4:
        display();
        break;
    case 5:
        printf(isEmpty() ? "Stack is empty\n" : "Stack is not empty\n");
        break;
    case 6:
        printf(isFull() ? "Stack is full\n" : "Stack is not full\n");
        break;
    case 7:
        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice! Try again.\n");
}
} while(choice != 7);

return 0;
```

```
}
```

Slip 21

Write a menu driven program for the following operations on singly linked list

- i. Insert an element in a linked list in a particular position.**
- ii. Search an element in a linked list.**

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Insert at given position (1-based)
Node* insertAtPosition(Node* head, int data, int pos) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;

    if(pos == 1) {
        newNode->next = head;
        return newNode;
    }

    Node* temp = head;
    for(int i = 1; i < pos-1 && temp != NULL; i++)
        temp = temp->next;

    if(temp == NULL) {
        printf("Position out of range!\n");
        free(newNode);
        return head;
    }

    newNode->next = temp->next;
    temp->next = newNode;
```

```

    return head;
}

// Search for an element
void searchElement(Node* head, int key) {
    Node* temp = head;
    int pos = 1;
    while(temp) {
        if(temp->data == key) {
            printf("Element %d found at position %d\n", key, pos);
            return;
        }
        temp = temp->next;
        pos++;
    }
    printf("Element %d not found\n", key);
}

```

```

// Display linked list
void displayList(Node* head) {
    if(head == NULL) {
        printf("List is empty\n");
        return;
    }
    Node* temp = head;
    while(temp) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

int main() {
    Node* head = NULL;
    int choice, data, pos;

    do {
        printf("\n--- Menu ---\n");
        printf("1. Insert element at position\n");
        printf("2. Search element\n");
    }
}

```

```

printf("3. Display list\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice) {
    case 1:
        printf("Enter element to insert: ");
        scanf("%d", &data);
        printf("Enter position to insert: ");
        scanf("%d", &pos);
        head = insertAtPosition(head, data, pos);
        break;
    case 2:
        printf("Enter element to search: ");
        scanf("%d", &data);
        searchElement(head, data);
        break;
    case 3:
        displayList(head);
        break;
    case 4:
        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice!\n");
}

} while(choice != 4);

return 0;
}

```

Slip 22

Q1) Write a menu driven program for the following operations on singly linked list.

- i. Insert an element in a linked list in a particular position.
- ii. Delete a particular element from a linked list.

```
#include <stdio.h>
```

```

#include <stdlib.h>

// Node structure
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Insert at a given position (1-based)
Node* insertAtPosition(Node* head, int data, int pos) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;

    if(pos == 1) {
        newNode->next = head;
        return newNode;
    }

    Node* temp = head;
    for(int i = 1; i < pos-1 && temp != NULL; i++)
        temp = temp->next;

    if(temp == NULL) {
        printf("Position out of range!\n");
        free(newNode);
        return head;
    }

    newNode->next = temp->next;
    temp->next = newNode;
    return head;
}

// Delete a particular element
Node* deleteElement(Node* head, int key) {
    Node* temp = head;
    Node* prev = NULL;

    while(temp != NULL && temp->data != key) {

```

```

    prev = temp;
    temp = temp->next;
}

if(temp == NULL) {
    printf("Element %d not found\n", key);
    return head;
}

if(prev == NULL) { // Element is at head
    head = head->next;
} else {
    prev->next = temp->next;
}

free(temp);
printf("Element %d deleted successfully\n", key);
return head;
}

// Display linked list
void displayList(Node* head) {
    if(head == NULL) {
        printf("List is empty\n");
        return;
    }
    Node* temp = head;
    while(temp) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    Node* head = NULL;
    int choice, data, pos;

    do {
        printf("\n--- Linked List Menu ---\n");

```

```

printf("1. Insert element at position\n");
printf("2. Delete element\n");
printf("3. Display list\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice) {
    case 1:
        printf("Enter element to insert: ");
        scanf("%d", &data);
        printf("Enter position to insert: ");
        scanf("%d", &pos);
        head = insertAtPosition(head, data, pos);
        break;
    case 2:
        printf("Enter element to delete: ");
        scanf("%d", &data);
        head = deleteElement(head, data);
        break;
    case 3:
        displayList(head);
        break;
    case 4:
        printf("Exiting program.\n");
        break;
    default:
        printf("Invalid choice!\n");
}
} while(choice != 4);

return 0; }

```

Slip 23

Perform the following operations using static implementation of stack

- i. Accept n integers from user and push it into the stack.**
- ii. Pop the element from the stack and display it (Use isEmpty())**

```

#include <stdio.h>
#define MAX 100

```

```

int stack[MAX];
int top = -1;

int isEmpty() {
    return top == -1;
}

void push(int value) {
    if (top == MAX - 1)
        printf("Stack Overflow!\n");
    else
        stack[++top] = value;
}

void pop() {
    if (isEmpty())
        printf("Stack Underflow!\n");
    else
        printf("%d\n", stack[top--]);
}

void main() {
    int n, i, value;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    // Push elements
    for (i = 0; i < n; i++) {
        scanf("%d", &value);
        push(value);
    }

    // Pop and display elements
    printf("Popping elements:\n");
    while (!isEmpty()) {
        pop();
    }
}

```

Accept the array of n integers from user and sort the array in ascending order by using recursive Counting sort algorithm

```
#include <stdio.h>
```

```
void countingSortRecursive(int arr[], int n, int maxVal);
```

```
void countingSortRecursive(int arr[], int n, int maxVal) {
```

```
    int count[maxVal + 1];
```

```
    for (int i = 0; i <= maxVal; i++) count[i] = 0;
```

```
    // Count frequency
```

```
    for (int i = 0; i < n; i++) count[arr[i]]++;
```

```
    // Recursive function to fill array
```

```
    void fill(int value, int *index) {
```

```
        if (value > maxVal) return;
```

```
        while (count[value] > 0) {
```

```
            arr[(*index)++] = value;
```

```
            count[value]--;
```

```
        }
```

```
        fill(value + 1, index);
```

```
    }
```

```
    int index = 0;
```

```
    fill(0, &index);
```

```
}
```

```
void main() {
```

```
    int n, i, maxVal = 0;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    printf("Enter elements: ");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
        if (arr[i] > maxVal) maxVal = arr[i];
```

```
    }
```

```

countingSortRecursive(arr, n, maxVal);

printf("Sorted array:\n");
for (i = 0; i < n; i++)
    printf("%d ", arr[i]);
}

```

Slip 25

Write a menu driven program for the following operations on singly linked list.

i. Create linked list and display it.

ii. Accept an element from user and concatenate it with a created link list.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Node structure
```

```
struct Node {
    int data;
    struct Node* next;
};
```

```
struct Node* head = NULL;
```

```
// Function to create linked list
```

```
void createList() {
    int n, i, value;
    struct Node *temp, *newNode;
```

```
    printf("Enter number of nodes: ");
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) {
        newNode = (struct Node*)malloc(sizeof(struct Node));
        printf("Enter element %d: ", i + 1);
        scanf("%d", &newNode->data);
        newNode->next = NULL;
```

```
        if (head == NULL)
            head = newNode;
        else {
```

```

        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
}
printf("Linked list created successfully.\n");
}

```

// Function to display linked list

```

void displayList() {
    struct Node* temp = head;
    if (temp == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("Linked list elements: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

```

// Function to concatenate a new element

```

void concatenateElement() {
    int value;
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    struct Node* temp;

    printf("Enter element to concatenate: ");
    scanf("%d", &value);
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
        head = newNode;
    else {
        temp = head;
        while (temp->next != NULL)

```

```

        temp = temp->next;
        temp->next = newNode;
    }

    printf("%d concatenated to the linked list.\n", value);
}

void main() {
    int choice;
    while (1) {
        printf("\n--- Menu ---\n");
        printf("1. Create linked list\n");
        printf("2. Display linked list\n");
        printf("3. Concatenate an element\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: createList(); break;
            case 2: displayList(); break;
            case 3: concatenateElement(); break;
            case 4: exit(0);
            default: printf("Invalid choice! Try again.\n");
        }
    }
}

```